

INFORMAÇÕES

MÓDULO	AULA	INSTRUTOR
08	ASP.NET Core Web API – Avançado – EF Core, LINQ e Segurança	06 – Exportação Dinâmica e a Camada CrossCutting
		Guilherme Paracatu

1. A Teoria da Arquitetura Transversal**1.1. O que é a camada CrossCutting?**

Na Clean Architecture, aprendemos que as camadas são empilhadas de forma rigorosa: A Apresentação (API) fala com a Aplicação, que fala com o Domínio, que fala com a Infraestrutura. Uma camada superior sempre depende da inferior, e o fluxo de dependência aponta para o centro.

Mas o que acontece quando precisamos de uma ferramenta que é útil para **todas** as camadas? Por exemplo:

- Geração de planilhas Excel ou PDFs.
- Formatação avançada de datas ou strings.
- Criptografia de senhas (Hashes).
- Envio de E-mails.

Se colocarmos um "Gerador de Excel" no Domínio, sujamos nossas regras de negócio com código de infraestrutura de arquivos. Se colocarmos na Controller, os nossos Handlers não conseguem acessá-lo.

É para resolver isso que existe a camada **Cross-Cutting (Transversal)**. Imagine o CrossCutting como um "canivete suíço" que fica de pé, cruzando todas as camadas do projeto.

Qualquer parte do sistema pode instanciar uma classe do CrossCutting, pois ela não contém regras de negócio exclusivas da Concessionária, apenas **Helpers (Ferramentas Utilitárias Genéricas)**.

1.2. Exportação Dinâmica de Dados (RBAC)

No mundo corporativo, a exportação de dados para Excel é o recurso mais requisitado pelas diretorias. Porém, entregar os mesmos dados para todos os funcionários é um risco de segurança.

Nós vamos aplicar o conceito de **Role-Based Access Control** integrado ao JWT. O sistema será inteligente o suficiente para:

1. Ler a "Pulseira VIP" (JWT) do usuário.
2. Identificar o cargo dele (Gerente ou Vendedor).
3. Montar uma planilha **dinâmica**. Se for Vendedor, a coluna "Preço de Custo" simplesmente não existirá no arquivo final.

2. A Prática: Implementando o Exportador em Clean Architecture

Passo 0: Atualizando o Domínio e o Banco de Dados

Antes de exportarmos os dados dinamicamente, percebemos que o nosso catálogo de veículos ainda não possui informações financeiras!

Como estamos trabalhando com **Domain-Driven Design (DDD)**, não vamos apenas adicionar as colunas. Vamos ensinar à nossa entidade Veiculo as regras de negócio financeiras (exemplo: o preço de venda nunca pode ser menor que o preço de custo).

1. Atualizando a Entidade: Abra o arquivo Veiculo.cs no projeto ITB.Domain/Entities e substitua pelo código abaixo. Repare na nova regra ValidarPrecos e no método AtualizarPrecos.

```
C#
using ITB.Domain.Core.Exceptions;
using ITB.Domain.Entities;

public class Veiculo
{
    public int Id { get; private set; }
    public string Modelo { get; private set; } = null!;
    public string Placa { get; private set; } = null!;
    public int Ano { get; private set; }
    public int MarcaId { get; private set; }

    // Novas propriedades financeiras
    public decimal PrecoCusto { get; private set; }
    public decimal PrecoVenda { get; private set; }
```

```

public bool Ativo { get; private set; } = true;

public virtual Marca Marca { get; private set; } = null!;

protected Veiculo() { }

public Veiculo(string modelo, string placa, int ano, int marcaId, decimal
precoCusto, decimal precoVenda)
{
    ValidarPlaca(placa);
    ValidarDados(modelo, ano);
    ValidarPrecos(precoCusto, precoVenda); // Acionando a nova regra

    if (marcaId <= 0) throw new DomainException("A Marca é obrigatória.");

    Modelo = modelo;
    Placa = placa;
    Ano = ano;
    MarcaId = marcaId;
    PrecoCusto = precoCusto;
    PrecoVenda = precoVenda;
}

// --- Comportamentos de Atualização (PUT) ---

public void AtualizarDados(string modelo, int ano)
{
    if (Modelo == modelo && Ano == ano) return;
    ValidarDados(modelo, ano);
    Modelo = modelo;
    Ano = ano;
}

// Novo comportamento financeiro
public void AtualizarPrecos(decimal precoCusto, decimal precoVenda)
{
    if (PrecoCusto == precoCusto && PrecoVenda == precoVenda) return;
    ValidarPrecos(precoCusto, precoVenda);
    PrecoCusto = precoCusto;
    PrecoVenda = precoVenda;
}

public void AlterarPlaca(string novaPlaca, bool placaJaExisteEmOutroVeiculo)
{
    if (Placa == novaPlaca) return;

    if (placaJaExisteEmOutroVeiculo)
        throw new DomainException("Esta placa já pertence a outro veículo.");
}

```

```

        ValidarPlaca(novaPlaca);
        Placa = novaPlaca;
    }

    public void AlterarMarca(int novaMarcaId, bool novaMarcaExiste)
    {
        if (MarcaId == novaMarcaId) return;

        if (novaMarcaId <= 0)
            throw new DomainException("A nova marca informada é inválida.");

        if (!novaMarcaExiste)
            throw new DomainException("A nova marca informada não existe no sistema.");

        MarcaId = novaMarcaId;
    }

    public void Desativar()
    {
        if (!Ativo) return;
        Ativo = false;
    }

    // --- Validações Internas ---

    private void ValidarPlaca(string placa)
    {
        if (string.IsNullOrEmpty(placa) || placa.Length != 7)
            throw new DomainException("Formato de placa inválido.");
    }

    private void ValidarDados(string modelo, int ano)
    {
        if (string.IsNullOrEmpty(modelo)) throw new DomainException("Modelo obrigatório.");
        if (ano < 1900 || ano > DateTime.Now.Year + 1) throw new DomainException("Ano inválido.");
    }

    // Nova blindagem de negócio
    private void ValidarPrecos(decimal precoCusto, decimal precoVenda)
    {
        if (precoCusto < 0)
            throw new DomainException("O preço de custo não pode ser negativo.");

        if (precoVenda < precoCusto)
            throw new DomainException("Prejuízo detectado: O preço de venda não pode ser menor que o preço de custo.");
    }

```

```
}  
}
```

Atenção: Como alteramos o construtor da Entidade adicionando os preços, você precisará atualizar o seu Handler de Criação (AdicionarVeiculoHandler ou similar) para passar esses dois novos valores na hora de instanciar um new Veiculo(...)!

1.2. Atualizando o Mapeamento do Entity Framework (Fluent API)

Como adicionamos propriedades novas à nossa entidade, precisamos ensinar ao Entity Framework como criar essas colunas no banco de dados PostgreSQL. Como se trata de valores financeiros, devemos ser rigorosos com a precisão das casas decimais.

Abra o arquivo VeiculoMapping.cs dentro do projeto ITB.Infrastructure/Persistence/Mappings e adicione as configurações de PrecoCusto e PrecoVenda:

```
C#  
using Microsoft.EntityFrameworkCore;  
using Microsoft.EntityFrameworkCore.Metadata.Builders;  
  
namespace ITB.Infrastructure.Persistence.Mappings;  
  
public class VeiculoMapping : IEntityTypeConfiguration<Veiculo>  
{  
    public void Configure(EntityTypeBuilder<Veiculo> builder)  
    {  
        builder.ToTable("veiculos");  
  
        builder.HasKey(v => v.Id);  
  
        builder.Property(v => v.Modelo)  
            .IsRequired()  
            .HasMaxLength(150)  
            .HasColumnType("varchar(150)");  
  
        builder.Property(v => v.Placa)  
            .IsRequired()  
            .HasMaxLength(7)  
            .HasColumnType("char(7)");  
  
        builder.Property(v => v.Ano)  
            .IsRequired();  
  
        // --- INÍCIO DAS NOVAS PROPRIEDADES FINANCEIRAS ---  
        builder.Property(v => v.PrecoCusto)  
            .IsRequired()  
            .HasPrecision(12, 2); // 12 dígitos no total, sendo 2 casas decimais  
  
        builder.Property(v => v.PrecoVenda)
```

```

        .IsRequired()
        .HasPrecision(12, 2);
// --- FIM DAS NOVAS PROPRIEDADES ---

// Mapeamento da FK (Foreign Key)
builder.Property(v => v.MarcaId)
    .IsRequired();

builder.Property(v => v.Ativo)
    .IsRequired()
    .HasDefaultValue(true);

// Filtro global: o EF Core sempre ignorará veículos desativados nas consultas
builder.HasQueryFilter(x => x.Ativo);
}
}

```

Nota: O uso do `.HasPrecision(12, 2)` é uma boa prática absoluta para sistemas financeiros no .NET. Isso diz ao banco de dados PostgreSQL para criar a coluna do tipo `numeric(12,2)`, garantindo que você pode armazenar valores de até 9.999.999.999,99 sem sofrer perda de centavos por problemas de arredondamento na memória.

2. Gerando a Migration e Atualizando o Banco de Dados: Como alteramos a nossa entidade, o banco de dados PostgreSQL precisa ser avisado:

Bash

```
dotnet ef migrations add AdicionandoPrecosVeiculo --project ITB.Infrastructure --
startup-project ITB.API
```

```
dotnet ef database update --project ITB.Infrastructure --startup-project ITB.API
```

Passo 1: O Pacote NuGet e a Escolha da Biblioteca

Antes de rodarmos o comando de instalação, é importante entendermos **por que** escolhemos essa ferramenta específica. O ecossistema .NET possui várias bibliotecas para manipulação de planilhas, mas a eleita para a nossa arquitetura corporativa é a **ClosedXML**.

Por que não usar as outras opções do mercado?

- **Microsoft.Office.Interop.Excel:** A abordagem mais antiga. Ela exige que o aplicativo do Microsoft Excel esteja fisicamente instalado no computador que roda o código.
O Problema: Servidores de nuvem modernos (na AWS, Azure, ou containers Docker com Linux) não têm o pacote Office instalado. Se você usar o Interop, sua API vai gerar um erro fatal em produção.
- **OpenXML SDK (Oficial da Microsoft):** É a biblioteca base que não exige o Office instalado. O problema é que ela é extremamente complexa (baixo nível). Criar uma

tabela com fundo colorido usando OpenXML exige dezenas de linhas de código difíceis de dar manutenção.

- **EPPlus:** Era a biblioteca mais famosa do .NET, mas recentemente mudou seu modelo de licenciamento. Se você usar o EPPlus atual em um software comercial sem pagar a licença, a empresa da qual você trabalha pode enfrentar problemas jurídicos.

A Grande Vantagem do ClosedXML: Ele funciona como um "tradutor elegante" construído por cima da complexidade do OpenXML SDK.

Ele permite manipular planilhas usando Orientação a Objetos de forma muito intuitiva, não exige o Excel no servidor, funciona em qualquer sistema operacional (Linux/Mac/Windows) e o mais importante: é **100% open-source e gratuito (Licença MIT) para uso comercial e acadêmico.**

 **Material de Apoio (Para ir além da aula):** O nosso *Helper* fará uma exportação limpa e formatada, mas o ClosedXML faz muito mais do que isso. Ele permite criar gráficos nativos, tabelas dinâmicas, fórmulas matemáticas embutidas (ex: =SOMA(A1:A10)) e formatação condicional direto pelo código C#. Se você tem interesse em descobrir novos recursos para enriquecer seus relatórios no futuro, acesse a **Documentação Oficial da Comunidade:**  <https://closedxml.readthedocs.io/>

Mão na Massa: Abra o terminal do VS Code, navegue até a pasta do projeto ITB.CrossCutting e rode o comando abaixo para baixar a biblioteca:

```
Bash
dotnet add package ClosedXML
```

Passo 2: O Canivete Suíço (CrossCutting)

Crie uma pasta chamada Helpers dentro do projeto ITB.CrossCutting e crie o arquivo ExcelExportHelper.cs.

```
C#
using ClosedXML.Excel;

namespace ITB.CrossCutting.Helpers;

public static class ExcelExportHelper
{
    public static byte[] ExportarParaExcel<T>(IEnumerable<T> dados, List<string>
colunasPermitidas, string nomePlanilha = "Relatorio")
    {
        //Abertura do arquivo em memória
        using var workbook = new XLWorkbook();
        var worksheet = workbook.Worksheets.Add(nomePlanilha);

        //Uso de Reflection para ler a classe
        var propriedades = typeof(T).GetProperties()
            .Where(p => colunasPermitidas.Contains(p.Name))
            .ToList();
```

```

//0 Índice 1 do Excel
for (int i = 0; i < propriedades.Count; i++)
{
    var celula = worksheet.Cell(1, i + 1);
    celula.Value = propriedades[i].Name;
    celula.Style.Font.Bold = true;
    celula.Style.Fill.BackgroundColor = XLColor.LightGray;
}

// 2. Preenchimento dos Dados
var linhaAtual = 2;
foreach (var item in dados)
{
    for (int i = 0; i < propriedades.Count; i++)
    {
        var valor = propriedades[i].GetValue(item);
        var celula = worksheet.Cell(linhaAtual, i + 1);

        if (valor != null)
        {
            // Passa o valor numérico real para o Excel
            celula.Value = XLCellValue.FromObject(valor);

            // Se o valor for dinheiro (decimal ou double), formata a célula na
            hora!
            if (valor is decimal || valor is double)
            {
                // As aspas duplas forçam o Excel a desenhar o "R$"
                Literalmente.
                // O padrão #,##0.00 diz ao Excel para colocar ponto de milhar
                e 2 casas decimais.
                celula.Style.NumberFormat.Format = "\"R$\" #,##0.00";
            }
        }
        else
        {
            celula.Value = "";
        }
    }
    linhaAtual++;
}

worksheet.Columns().AdjustToContents();

//A Mágica do MemoryStream
using var stream = new MemoryStream();
workbook.SaveAs(stream);
return stream.ToArray();

```



```
}  
}
```

🔍 Entendendo o Código Linha a Linha

1. **using var workbook:** A palavra using é obrigatória aqui. Trabalhar com arquivos consome muita memória RAM. O using garante que, assim que a função terminar, o .NET vai acionar o *Garbage Collector* e destruir a planilha da memória, evitando que o servidor trave.
2. **Reflection (typeof(T).GetProperties()):** O .NET tem a capacidade de inspecionar a si mesmo durante a execução. O código olha para a classe (Veiculo, por exemplo) e descobre automaticamente quais são os nomes de suas propriedades. Depois, filtramos apenas as propriedades que o nosso Handler permitiu que fossem para o Excel.
3. **Índice i + 1:** Na programação C#, arrays e listas começam no índice 0. Porém, as linhas e colunas do Excel começam no número 1. Por isso, sempre somamos +1 na hora de dizer ao ClosedXML em qual célula ele deve escrever.
4. **MemoryStream:** Nós NÃO salvamos o arquivo no disco rígido do servidor (C:\temp). Servidores modernos ficam na nuvem (AWS/Azure) e podem não ter disco disponível ou dar erro de permissão. Nós criamos um "arquivo fantasma" na Memória RAM (MemoryStream), transformamos ele em um monte de bytes (ToArray()) e mandamos isso diretamente pela internet para o navegador do usuário fazer o download.

Passo 3: Isolando a Interface (O DTO de Resposta)

Crie no projeto ITB.Application/Dtos a classe que tráfegará os bytes do arquivo:

```
C#  
namespace ITB.Application.Dtos;  
  
public class ArquivoExportacaoDto  
{  
    public byte[] Conteudo { get; set; } = Array.Empty<byte>();  
    public string NomeArquivo { get; set; } = string.Empty;  
    public string ContentType { get; set; } = string.Empty;  
}
```

Passo 4: O Fluxo CQRS (A Query e o Handler)

Crie no projeto ITB.Application/Queries o arquivo:

```
C#  
namespace ITB.Application.Queries;  
  
public class ExportarVeiculosExcelQuery  
{  
    public string CargoUsuarioLogado { get; set; } = string.Empty;  
}
```

Crie o Handler no projeto ITB.Application/Handlers:

```
C#
using ITB.Application.Dtos;
using ITB.Application.Queries;
using ITB.CrossCutting.Helpers;
using ITB.Domain.Interfaces;

namespace ITB.Application.Handlers;

public class ExportarVeiculosExcelQueryHandler
{
    private readonly IVeiculoQuery _veiculoQuery;

    public ExportarVeiculosExcelQueryHandler(IVeiculoQuery veiculoQuery)
    {
        _veiculoQuery = veiculoQuery;
    }

    public async Task<ArquivoExportacaoDto> Handle(ExportarVeiculosExcelQuery request)
    {
        var veiculos = await _veiculoQuery.ObterTodos();

        // Os nomes aqui DEVEM ser idênticos aos nomes das propriedades do seu
        VeiculoDto
        var colunas = new List<string>
        {
            "Id",
            "Modelo",
            "Placa",
            "Ano",
            "PrecoVenda"
        };

        // Regra de Negócio Dinâmica: O Gerente vê o custo!
        if (request.CargoUsuarioLogado == "Gerente")
        {
            colunas.Add("PrecoCusto");
        }

        var bytesExcel = ExcelExportHelper.ExportarParaExcel(veiculos, colunas,
"Estoque");

        return new ArquivoExportacaoDto
        {
            Conteudo = bytesExcel,
            NomeArquivo = $"Relatorio_Veiculos_{DateTime.Now:ddMMyyyy}.xlsx",
        }
    }
}
```

```

        //O MIME Type
        ContentType = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    };
}
}

```

MIME Type: Esse texto gigante no ContentType é a forma padrão da internet dizer ao Google Chrome: *"Ei navegador, esse monte de bytes que eu estou te enviando é uma planilha do Excel. Pode abrir a caixinha de download com o ícone verde do Office!"* Se colocarmos errado, o navegador vai baixar um arquivo em branco.

Passo 5: A Injeção de Dependência (Não esqueça!)

Se não fizermos isso, o .NET vai dar um erro 500 dizendo que não sabe quem é o nosso Handler. Abra o projeto ITB.IoC, vá no arquivo onde vocês configuram as injeções e adicione:

```

C#
// Registrando o nosso novo Handler no contêiner de dependências do .NET
services.AddScoped<ExportarVeiculosExcelQueryHandler>();

```

Passo 6: A Controller

Abra a VeiculoController.cs:

```

C#
using ITB.Application.Queries;
using ITB.Application.Handlers;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace ITB.API.Controllers;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class VeiculoController : ControllerBase
{
    private readonly ExportarVeiculosExcelQueryHandler _handler;

    public VeiculoController(ExportarVeiculosExcelQueryHandler handler)
    {
        _handler = handler;
    }

    [HttpGet("exportar-excel")]
    public async Task<IActionResult> ExportarExcel()
    {
        var cargo = User.FindFirstValue(ClaimTypes.Role) ?? "Cliente";
    }
}

```

```

var query = new ExportarVeiculosExcelQuery { CargoUsuarioLogado = cargo };

var arquivoDto = await _handler.Handle(query);

// O método File() pega nossos bytes e empacota no formato exato que o HTTP
exige para downloads.
return File(arquivoDto.Conteudo, arquivoDto.ContentType,
arquivoDto.NomeArquivo);
}
}

```

🔗 Teste de Validação

1. Compile e execute a aplicação.
2. Faça o Login gerando um token para o usuário com perfil **Vendedor**.
3. Autentique o Swagger (botão Authorize) com este token.
4. Execute a rota GET /api/veiculo/exportar-excel e baixe a planilha. Ao abrir o arquivo, a coluna PrecoCusto **não existirá**.
5. Repita o processo com o token do **Gerente**. A coluna PrecoCusto aparecerá no final da tabela.

📋 Desafio Prático: O Relatório de Lucratividade

A História (Requisito de Negócio): A diretoria da concessionária solicitou uma melhoria no relatório. Precisamos incluir uma coluna '**Lucro**' **calculada**, eles querem que a exportação seja isolada da listagem comum da tela. O objetivo é que o Excel tenha sua própria estrutura de dados, independente do que aparece no site.

O Desafio:

1. **Criação do DTO de Exportação:** No projeto ITB.Application/Dtos, crie uma nova classe chamada VeiculoExportacaoExcelDto. Ela deve conter:
 - Id, Modelo, Placa, Ano, Marca (string), PrecoCusto, PrecoVenda e a nova propriedade Lucro.
2. **Ajuste na Query de Infraestrutura:** No seu projeto de Infrastructure, crie um novo método de busca para que ele retorne uma lista do novo VeiculoExportacaoExcelDto.
3. **A Permissão no Handler:** No ExportarVeiculosExcelQueryHandler, certifique-se de que a string "Lucro" e "PrecoCusto" só sejam adicionadas à lista de colunas se o cargo logado for "Gerente".
4. **Teste de Isolamento:** Adicione um campo DataCadastro apenas no DTO do Excel. Verifique se ele aparece na planilha sem "sujar" o JSON da sua rota de listagem (GET) normal. Pode colocar DateTime.Now apenas para similar.